

Proponer un circuito de decodificación que permita obtener un mapa de memoria ARC con 1 GByte instalado. Se cuenta con un chip de 512 MBytes y 2 chips de 256MBytes. Indicar todas las conexiones entre bus, memoria y procesador.

a) Cantidad de Circuitos y Cuales se van a usar

1º) Recordar los sufijos Kb, Mb, Gb

$$- 1Kb = 2^{10} = 1024$$

$$- 1Mb = 1Kb \times 1Kb = 2^{10} \cdot 2^{10} = 2^{20}$$

$$- 1Gb = 1Kb \times 1Mb = 2^{10} \cdot 2^{20} = 2^{30}$$

2º) Elegimos la cantidad de Circuitos integrados que vamos a usar:

$$- \text{Memoria de } 1\text{Gbyte} = 512\text{MB} + 2 \times 256\text{MB}$$

$$\text{IC1 } 512\text{MBytes} = 2^9 \times 2^{20} = 2^{29} \rightarrow \text{AO hasta A28}$$

$$\text{IC2 } 256\text{MBytes} = 2^8 \times 2^{20} = 2^{28} \rightarrow \text{AO hasta A27}$$

$$\text{IC2 } 256\text{MBytes} = 2^8 \times 2^{20} = 2^{28} \rightarrow \text{AO hasta A27}$$

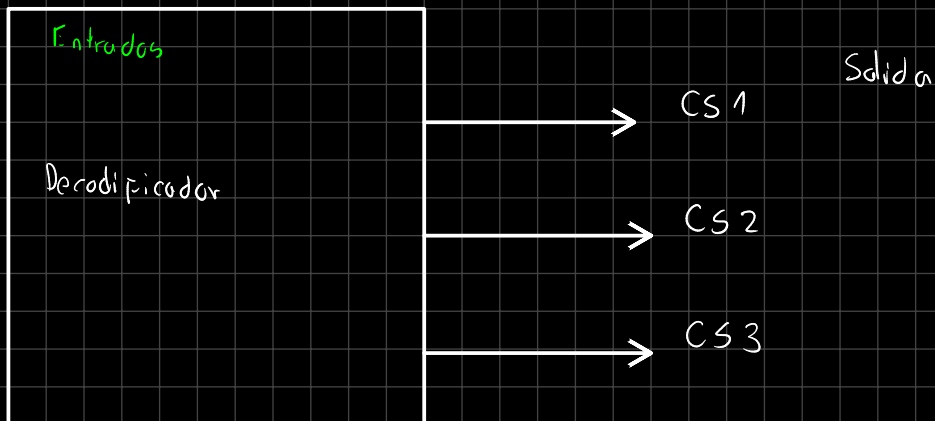
2º) Vamos a armar el Mapa de Memoria con la primera y ultima dirección.

Como trabajamos con $1\text{GByte} = 2^{30}$ entonces AO hasta A29

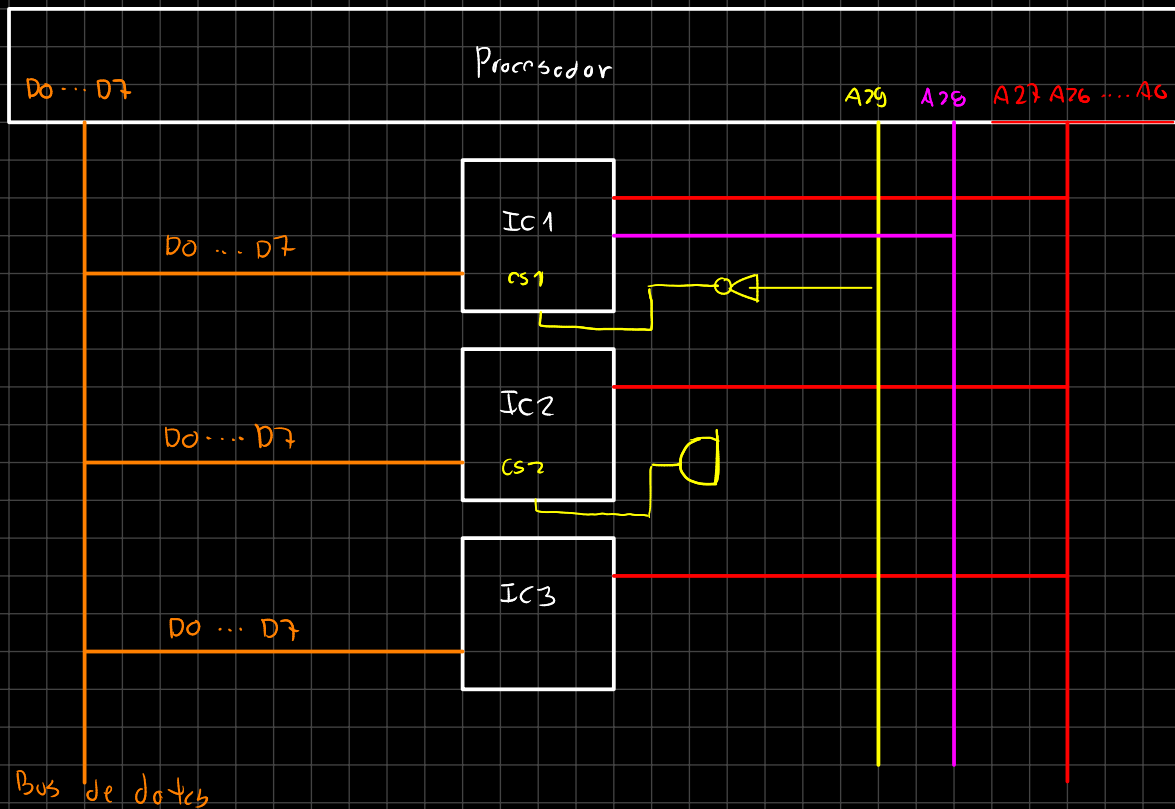
A29	A28	A27	A26	A25	...	AO		
0	0	0	0	0	...	0	1º Dirección	IC1
0	1	1	1	1	...	1	Ult. Dirección	IC1
1	0	0	0	0	...	0	1º Dirección	IC2
1	0	1	1	1	...	1	Ult. Dirección	IC2
1	1	0	0	0	...	0	1º Dirección	IC3
1	1	1	1	1	...	1	Ult Dirección	IC3

c) Decodificar los selectores del integrado (Chip Select)

	A29	A28	A27	A26	A25	
IC1	0	x	x	x	x	CS1 (Chip Select 1) "Dir que comienza con 0"
IC2	1	0	x	x	x	CS2 (Chip Select 2) "Dir que comienza con 10"
IC3	1	1	x	x	x	CS3 (Chip Select 3) "Dir que comienza con 11"



d) Dibujar el circuito



7. Diseñar un banco de memoria RAM de 1 GByte. Se dispone de un circuito integrado de 512 M palabras de 4 bits y dos de 256 M palabras de 4 bits. Se pide:
- Indicar cuántos y cuáles circuitos integrados se van a usar.
 - Armar un mapa de memoria en el que se vea la primera dirección y la última para cada circuito integrado.
 - Decodificar los selectores de integrado (Chip Select).
 - Dibujar el circuito.

a) 1º) Recordar las unidades Básicas

- $1KB = 2^{10}$
- $1MB = 2^{20}$
- $1GB = 2^{30}$

Direccionamiento

1x 512MB x 4 bits (D0...D3)

1x 512MB x 4 bits (D3...D7)

2x 256MB x 4 bits (D0...D3)

2x 256MB x 4 bits (D3...D7)

A

IC1 256MB = $2^8 \times 2^{20} = 2^{28} \Rightarrow$ va desde A0 hasta A27

IC2 256MB = $2^8 \times 2^{20} = 2^{28} \Rightarrow$ va desde A0 hasta A27

IC3 512MB = $2^9 \times 2^{20} = 2^{29} \Rightarrow$ va desde A0 hasta A28

B

IC1 256MB = $2^8 \times 2^{20} = 2^{28} \Rightarrow$ va desde A0 hasta A27

IC2 256MB = $2^8 \times 2^{20} = 2^{28} \Rightarrow$ va desde A0 hasta A27

IC3 512MB = $2^9 \times 2^{20} = 2^{29} \Rightarrow$ va desde A0 hasta A28

b)

Banco de Memoria

A29 A28 A27 A26 A25 A0

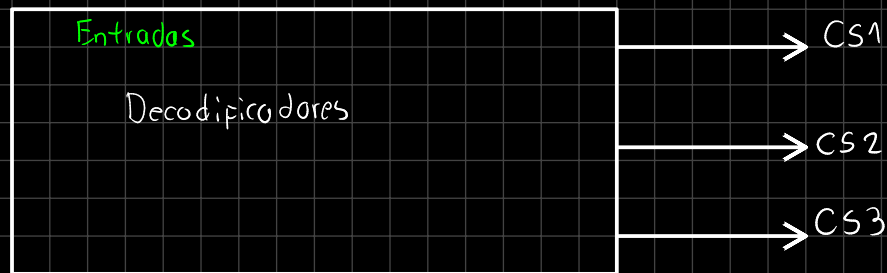
0	0	0	0	0	...	0	1º Dir IC1
0	0	1	1	1	...	1	Ult. Dir IC1

0	1	0	0	0	...	0	1º Dir IC2
0	1	1	1	1	...	1	Ult. Dir IC2

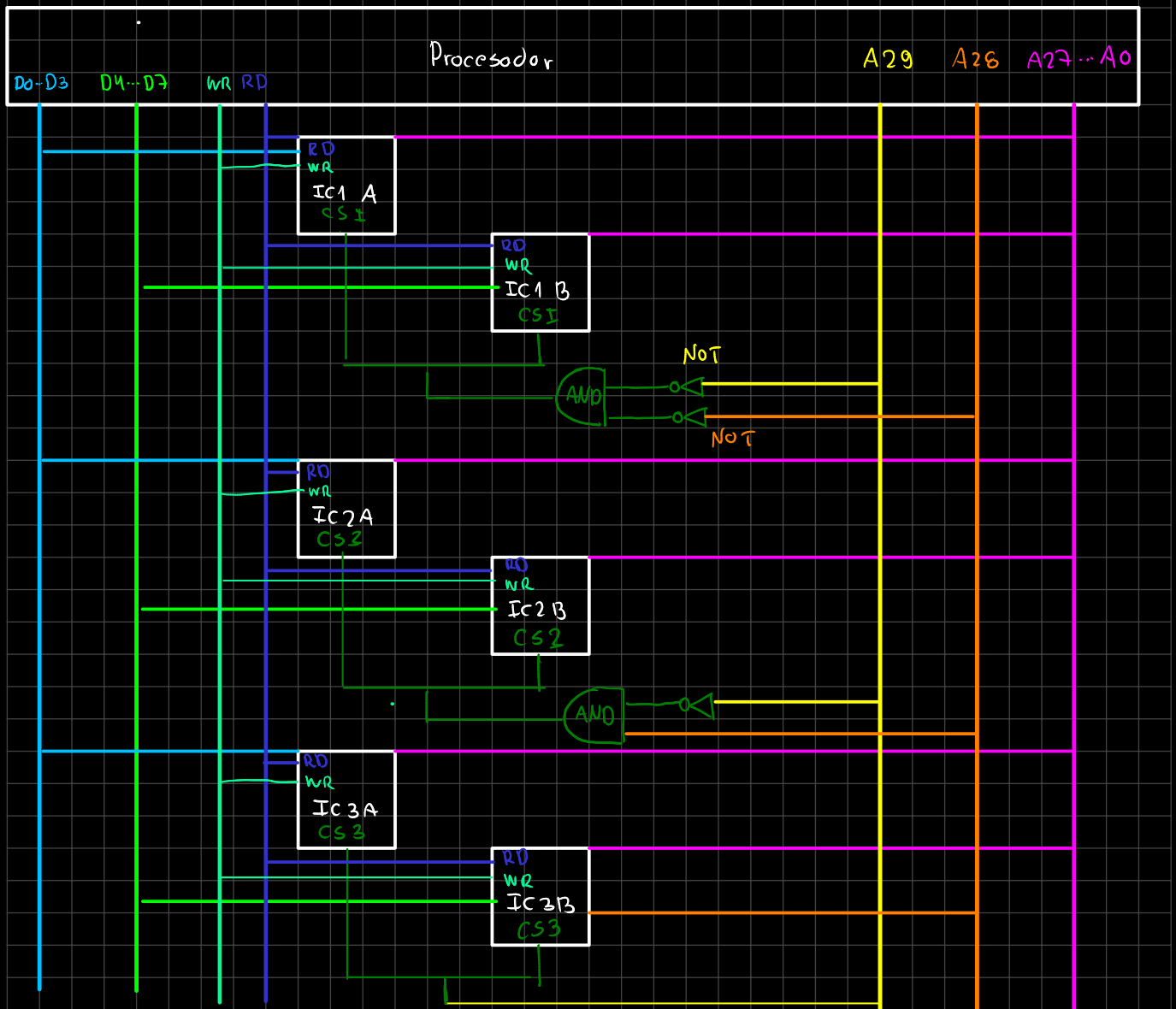
1	0	0	0	0	...	0	1º Dir IC3
1	1	1	1	1	...	1	Ult. Dir IC3

c) Decodificar los selectores de integrados (Chip Select)

	A29	A28	A27	A26		A0
Ic1	0	0	X	X		X
Ic2	0	1	X	X		X
Ic3	1	X	X	X		X



d) Dibujar el Circuito



Bus de datos

Bus de Direcciones

Proponer un circuito de decodificación que permita obtener un mapa de memoria ARC con 1 GByte instalado. Se cuenta con un chip de 512 MBytes y 2 chips de 256MBytes. Indicar todas las conexiones entre bus, memoria y procesador.

a) Cantidad de Circuitos y Cuales se van a usar

1º) Recordar los Sufijos KB, MB, GB

$$- 1 \text{ KB} = 2^{10}$$

$$- 1 \text{ MB} = 1 \text{ KB} \times 1 \text{ KB} = 2^{10} \times 2^{10} = 2^{20}$$

$$- 1 \text{ GB} = 1 \text{ KB} \times 1 \text{ KB} \times 1 \text{ KB} = 2^{10} \times 2^{10} \times 2^{10} = 2^{30}$$

2º) Elegimos la cantidad de circuitos integrados que vamos a usar.

$$- \text{Memoria de 1 GBytes} = 512 \text{ MB} + 2 \times 256 \text{ MB}$$

$$\text{IC1 } 512 \text{ MB} = 2^9 \times 2^{20} = 2^{29} \rightarrow \text{A0 hasta A28}$$

$$\text{IC2 } 256 \text{ MB} = 2^8 \times 2^{20} = 2^{28} \rightarrow \text{A0 hasta A27}$$

$$\text{IC3 } 256 \text{ MB} = 2^8 \times 2^{20} = 2^{28} \rightarrow \text{A0 hasta A27}$$

b) Vamos a armar un mapa de Memoria con la primera y ultima dirección.

Como Trabajamos con $1 \text{ GB} = 2^{30}$ entonces va desde A0 hasta A29

A29 A28 A27 A26 A25 A24 A0

0	0	0	0	0	0		0	1º Dirección	IC1
0	1	1	1	1	1		1	Ult. Dirección	IC1

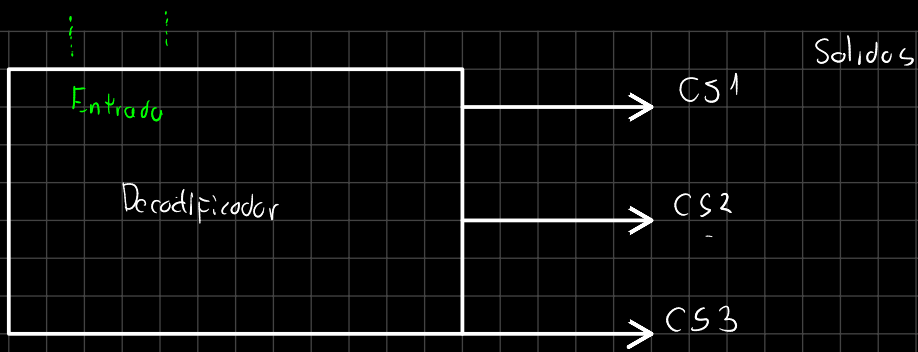
1	0	0	0	0	0		0	1º Dirección	IC2
1	0	1	1	1	1		1	Ult. Dirección	IC2

1	1	0	0	0	0		0	1º Dirección	IC3
1	1	1	1	1	1		1	Ult. Dirección	IC3

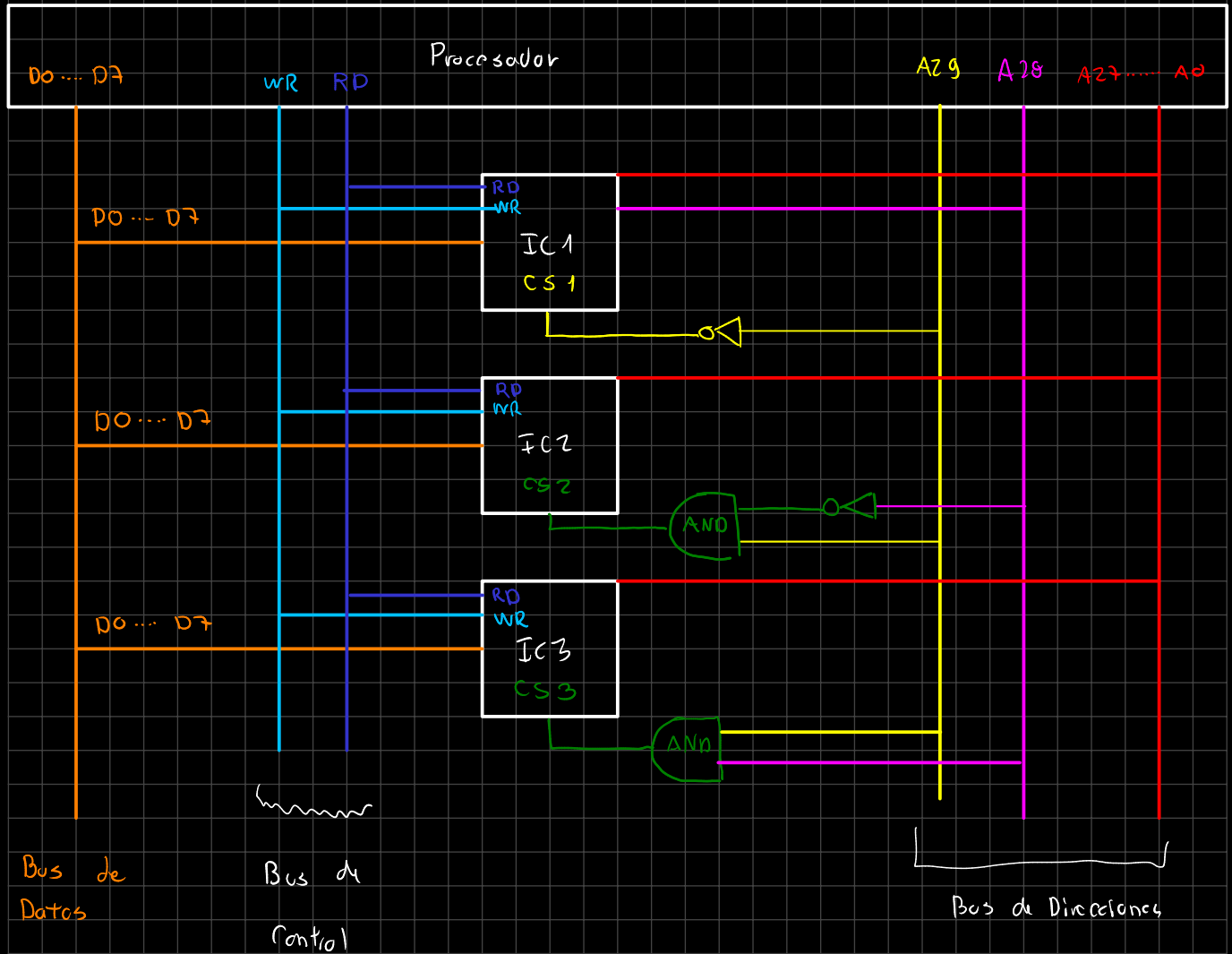
c) Decodificar los selectores del integrado (Chip Select)

A29 A28 A27 A26 A0

IC1	0	X	X	X		X	CS1 (Chip Select 1)
IC2	1	0	X	X		X	CS2 (Chip Select 2)
IC3	1	1	X	X		X	CS3 (Chip Select 3)



d) Dibujar el Circuito



8. Diseño de un Banco de Memoria Diseñar un sistema de memoria RAM de 256 KB utilizando circuitos integrados de 64 KB y 128 KB. a) Especificar cuántos circuitos de cada tipo se necesitan. b) Dibujar un mapa de memoria indicando las direcciones de inicio y fin de cada integrado. c) Diseñar la lógica de selección de chips. d) Representar el circuito esquemáticamente.

a) Cantidad de Circuitos y cuales se van a usar

1º) Recordar las siglas: KB, MB, GB.

- KB = 2^{10}

- MB = KB x KB = $2^{10} \times 2^{10} = 2^{20}$

- GB = KB x KB x KB = $2^{10} \times 2^{10} \times 2^{10} = 2^{30}$

2º) Elegimos la cantidad de circuitos Integrados que vamos a usar

- Memoria de 256 KB $\Rightarrow 2^8 \times 2^{10} = 2^{18}$ van desde A0 hasta 17

IC1 64 KB = $2^6 \times 2^{10} = 2^{16}$ de A0 hasta A15

IC2 64 KB = $2^6 \times 2^{10} = 2^{16}$ de A0 hasta A15

IC3 128 KB = $2^7 \times 2^{10} = 2^{17}$ de A0 hasta A16

b) Mapa de Memoria

A17 A16 A15 A14 A0

0	0	0	0		0	1º Dir IC1
0	0	1	1		1	Ult Dir IC1

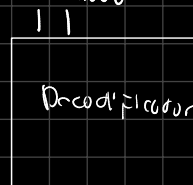
0	1	0	0		0	1º Dir IC2
0	1	1	1		1	Ult Dir IC2

1	0	0	0		1	1º Dir IC3
1	1	1	1		1	Ult Dir IC3

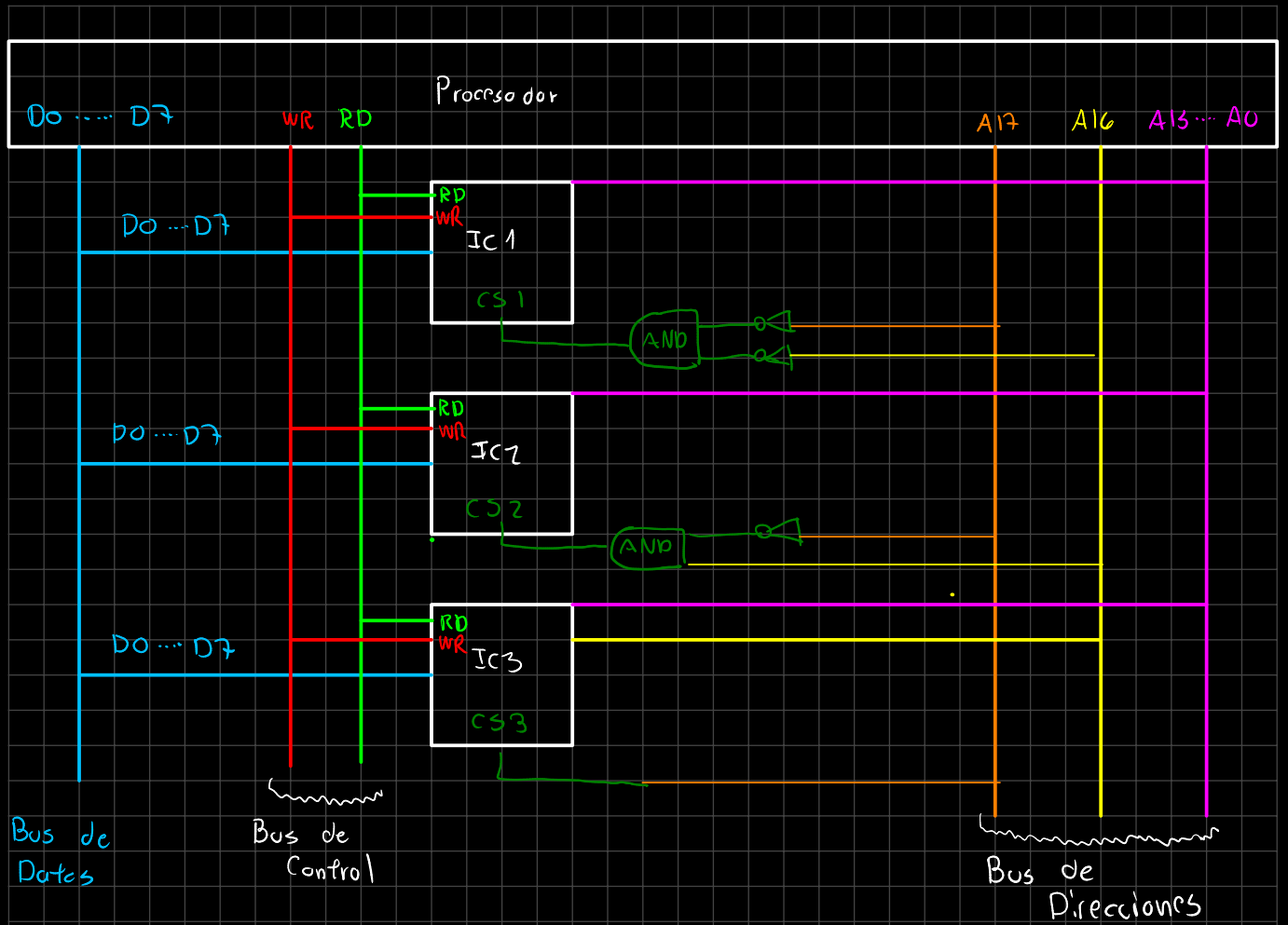
c) A17 A16 A15 A14 A0

IC1	0	0	x	x	x
IC2	0	1	x	x	x
IC3	1	x	x	x	x

Entrada



Salida
cs1
cs2
cs3



8. **Diseño de un Banco de Memoria** Diseñar un sistema de memoria RAM de 256 KB utilizando circuitos integrados de 64 KB y 128 KB. a) Especificar cuántos circuitos de cada tipo se necesitan. b) Dibujar un mapa de memoria indicando las direcciones de inicio y fin de cada integrado. c) Diseñar la lógica de selección de chips. d) Representar el circuito esquemáticamente.

a) 1º) Analizo los sufijos KB, MB, GB

$$- \text{KB} = 2^{10}$$

$$- \text{MB} = 2^{20}$$

$$- \text{GB} = 2^{30}$$

2º) Elijo la cantidad de chips a usar

$$256 \text{ KB} = 2^8 \times 2^{10} = 2^{18} \quad \text{de } A_0 \text{ hasta } A_{17}$$

$$\text{IC1} \quad 128 \text{ KB} = 2^7 \times 2^{10} = 2^{17} \quad \text{va de } A_0 \text{ hasta } A_{16}$$

$$\text{IC2} \quad 64 \text{ KB} = 2^6 \times 2^{10} = 2^{16} \quad \text{va de } A_0 \text{ hasta } A_{15}$$

$$\text{IC3} \quad 64 \text{ KB} = 2^6 \times 2^{10} = 2^{16} \quad \text{va de } A_0 \text{ hasta } A_{15}$$

b) Mapa de Memoria

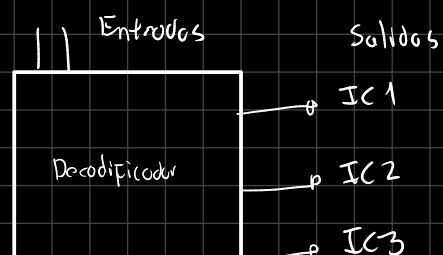
A17	A16	A15	A14	A13	...	A0	
0	0	0	0	0	...	0	1º Dirección IC1
0	1	1	1	1	...	1	Última Dirección IC1

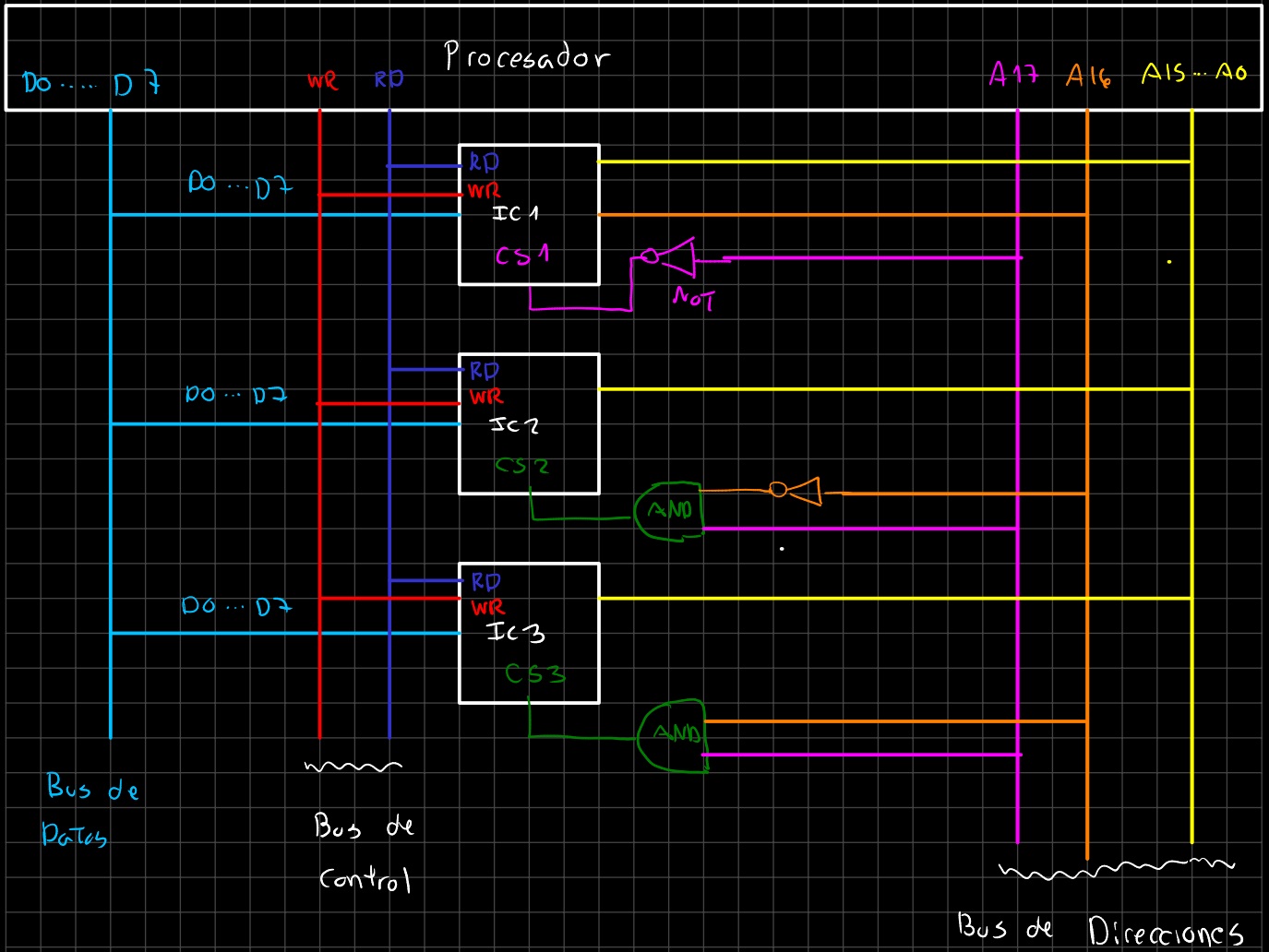
1	0	0	0	0	...	0	1º Dirección IC2
1	0	1	1	1	...	1	Últ. Dirección IC2

1	1	0	0	0	...	0	1º Dirección IC3
1	1	1	1	1	...	1	Últ. Dirección IC3

c)

	A17	A16	A15	A14
IC1	0	x	x	x
IC2	1	0	x	x
IC3	1	1	x	x





3. **Operaciones en una Máquina Elemental** Si el bit 4 de un valor almacenado en la dirección C5 es 0, intercambiar los bits 0 y 1 del mismo valor y almacenar el resultado en la dirección C6. En caso contrario, forzar a "1" el bit 2 y a "0" el bit 3 sin modificar los demás bits. El programa comienza en la dirección 70

Detección: valor almacenado en CS

70		2	0	0	0
72		1	1	C	5
74		2	2	2	0
76		8	3	1	2
78		B	3	8	6

AN_0

X	X	X	X	X	X	X	X
0	0	1	0	0	0	0	0
0	0	X	0	0	0	0	0

 $\rightarrow$

0	0
2	0

Caso	del	No	V
7A	2	4	7
7C	2	5	4
7E	8	6	4
80	7	7	5
02	3	7	5
04	10	8	5

AND

X	X	X	Y	X	X	X	X
1	1	1	1	0	1	1	1
OR				X	X	X	X
X	X	Y	Y	0	X	X	X
0	0	0	0	0	1	0	0
X	X	Y	X	0	1	X	X

→ F7h

→ 04h

Cosa del sì				non			
				X	X	X	X
86	2	4	03		X	X	X
88	9	5	14		0	0	0
8A	3	5	C6		0	0	1
8C	C	0	00		X	X	X

2. **IEEE 754** Convertir de punto flotante simple precisión a decimal, explicando cada paso, el número punto flotante A56DFB00. ¿Punto flotante tiene más precisión que punto fijo?

A	5	6	D	F	B	O	O	h
---	---	---	---	---	---	---	---	---

I) Converter a binario

1 0 1 0 0 1 0 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0

II) Grilla

Signo: $1 \rightarrow$ Negativo

Exponente: 0 1 0 0 1 0 1 0

Montiza: 1. 1 1 0 1 1 0 1 1 1 1 1 0 1 1

III) Exponente en Exceso

0	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---

$$2^6 + 2^3 + 2^1 = 74 \quad \begin{array}{l} \text{quitarles} \\ \text{el exceso} \\ 127 \end{array} \Rightarrow 74 - 127 = -53$$

IV) Convertir a Decimal la montiza

5	-1	-7		-4	-5	-7	-9	-9	-10	-11	-12		-14	-15
1	1	1	0	1	1	0	1	1	1	1	1	1	0	1

1 1 1 0 1 1 0 1 1 1 1 1 0 1 \

$$2^0 + 2^{-1} + 2^{-2} + 2^{-4} + 2^{-5} + 2^{-7} + 2^{-8} + 2^{-9} + 2^{-10} + 2^{-11} + 2^{-12} + 2^{-14} + 2^{-15}$$

1.859222412

b) Juntao Todo

$$- [1.859222412] \times 2^{-53}$$

$$-2.06415153 \times 10^{-16}$$

El punto Flotante ofrece un Rango Dinámico Mucho Mayor y ofrece una

Precisión Relativa (El error es Proporcional al número)

1. **Sistemas Numéricos** En una ALU de 8 bits, realizar las siguientes operaciones en binario con operandos representados como números binarios enteros con signo. Indicar el resultado y verificar si hay desbordamiento. Escribir el valor de los flags CVZNP. a) 1111000 - 0000101 b) FE - FD

$$\begin{array}{r} \text{a)} \quad - \quad 1111000 \\ \quad \quad 0000101 \\ \hline \\ \quad \quad \begin{array}{r} \\ \\ \\ \\ \\ \\ \\ \end{array} \\ + \quad \begin{array}{r} 1111000 \\ 1111011 \end{array} \\ \hline 11110011 \end{array}$$

$$C=1 \quad V=0 \quad Z=0 \quad N=1 \quad P=1$$

Dado que se trata con signo podemos decir que influye "V"

Si $V=0 \Rightarrow$ No hay desbordamiento.

$$\begin{array}{r} \text{b)} \quad - \quad \text{FE} \\ \quad \quad \text{FD} \\ \hline \\ \quad \quad \begin{array}{r} \\ \\ \\ \\ \\ \\ \\ \end{array} \\ + \quad \begin{array}{r} 11111110 \\ 00000011 \end{array} \\ \hline 10000000 \end{array}$$

$$C=1 \quad V=0 \quad Z=0 \quad N=0 \quad P=1$$

Dado que también se lo trata con signo nos fijamos en V si $V=0$

No hay desbordamiento

Si $V=1$ Hay desbordamiento.

Macomos Complemento a la Base

$$\begin{array}{r} + \quad 1111010 \\ \\ \\ \\ \\ \\ \\ \end{array}$$

Complemento a la base

$$\begin{array}{r} 1111101 \\ + \quad 00000010 \\ \hline 00000011 \end{array}$$

Resumen:

En números de Representación [con signo] influye V $\rightarrow V=0$ No hay desbordamiento.
 $\rightarrow V=1$ Hay desbordamiento.

En números de representación [sin signo] influye C $\left[\begin{array}{l} \text{si } C=0 \text{ No hay desbordamiento} \\ \text{si } C=1 \text{ Hay desbordamiento.} \end{array} \right.$

1. **Sistemas Numéricos** En una ALU de 8 bits, realizar las siguientes operaciones en binario con operandos representados como números binarios enteros con signo. Indicar el resultado y verificar si hay desbordamiento. Escribir el valor de los flags CVZNP. a) 1111000 - 00000101 b) FE - FD
2. **IEEE 754** Convertir de punto flotante simple precisión a decimal, explicando cada paso, el número punto flotante A56DFB00. ¿Punto flotante tiene más precisión que punto fijo?
3. **Operaciones en una Máquina Elemental** Si el bit 4 de un valor almacenado en la dirección C5 es 0, intercambiar los bits 0 y 1 del mismo valor y almacenar el resultado en la dirección C6. En caso contrario, forzar a "1" el bit 2 y a "0" el bit 3 sin modificar los demás bits. El programa comienza en la dirección 70
4. **ISA x86** A partir de la dirección 0200 hay un número de 3 bytes de longitud. En la dirección de la memoria que está después de ese número, hay otro número de 24 bits. Sumar los dos números y guardar el resultado a partir de la dirección posterior al segundo número.
5. **Ensamblador ARM** Escribir el número 5 en un display 7 segmentos mapeado en la dirección 0xFF200020 de la memoria. La barra superior del display tiene asignado el número 0, la numeración de las barras crece en sentido horario y la barra horizontal del medio es el número 6.
6. **Arquitectura y Procesadores** Indicar si las siguientes afirmaciones son Verdaderas o Falsas, justificando cada respuesta:
Arquitectura:
 - a. Las instrucciones en CISC no son uniformes en tamaño y no se ejecutan en ciclos fijos.
 - b. Los procesadores RISC, al tener menos instrucciones y una menor complejidad en su decodificación y ejecución, generalmente consumen menos energía para ejecutar las mismas tareas en comparación con los procesadores CISC.**Pipeline:**
 - c. Pipeline es una técnica de diseño utilizada para mejorar la velocidad de procesamiento al permitir que varias instrucciones se ejecuten de manera secuencial (es decir, una después de la otra) en diferentes etapas del proceso de ejecución.

- d. Un riesgo en pipeline es cuando una instrucción depende del resultado de una instrucción anterior que aún no ha terminado de ejecutarse.
- Performance:**
- e. Un aumento en la frecuencia de reloj de un procesador siempre resulta en un aumento proporcional en el valor de MIPS, manteniendo constante el número de instrucciones por ciclo (IPC).
 - f. Para los siguientes datos la computadora A tiene mejor performance que la computadora B porque ejecuta menos instrucciones en el mismo tiempo. Datos de la computadora A: Frecuencia de reloj: 3 GHz, CPI (Ciclos por Instrucción): 1,5. Datos de la computadora B: Frecuencia de reloj: 2,5 GHz, CPI (Ciclos por Instrucción): 1,0.

7. Componentes - Memoria Caché

- a. ¿Cuál es la diferencia principal entre la memoria RAM y la memoria caché en términos de velocidad y propósito dentro de un sistema computacional?
- b. ¿De qué manera la latencia y el ancho de banda afectan el rendimiento de la memoria RAM en comparación con la memoria caché?
- c. ¿Qué es la "localidad" en el contexto de la memoria caché?

8. **Diseño de un Banco de Memoria** Diseñar un sistema de memoria RAM de 256 KB utilizando circuitos integrados de 64 KB y 128 KB. a) Especificar cuántos circuitos de cada tipo se necesitan. b) Dibujar un mapa de memoria indicando las direcciones de inicio y fin de cada integrado. c) Diseñar la lógica de selección de chips. d) Representar el circuito esquemáticamente.

7. Componentes - Memoria Caché

a. ¿Cuál es la diferencia principal entre la memoria RAM y la memoria caché en términos de velocidad y propósito dentro de un sistema computacional?

b. ¿De qué manera la latencia y el ancho de banda afectan el rendimiento de la memoria RAM en comparación con la memoria caché?

c. ¿Qué es la "localidad" en el contexto de la memoria caché?

7a)	Memoria RAM	Memoria Caché
Velocidad	Relativamente más lenta comparado con el cache	Es mucho más rápida, solo después de los Registros de una CPU.
Propósito	Almacenar Temporalmente los datos y programas que el CPU está utilizando o necesita pronto	Almacenar una pequeña porción de los datos y las instrucciones más frecuentes usadas que residen en la RAM. Su objetivo es reducir el tiempo que el CPU debe esperar por los datos actuando como un intermediario ultrarápido.
Tamaño Típico	Grande (GB)	Pequeño (KB o MB)

b) Latencia:

- RAM: La latencia de la RAM es significativamente mayor que la de la cache. Una latencia alta significa que el CPU debe esperar más ciclos de reloj para que los datos sean recuperados de la RAM. Esto es un Factor limitante en el rendimiento General del Sistema.

- Cache: La cache está diseñada para tener una latencia extremadamente baja. Esto minimiza el tiempo de espera de la CPU, permitiendo que las operaciones se realicen con mucho más rapidez, mejorando drásticamente el rendimiento cuando los datos requeridos están en la cache.

Ancho de Banda:

Ram: El ancho de banda de la RAM es el volumen total de datos que puede transferir en un periodo de tiempo. Si bien la RAM moderna tiene un ancho de banda alto, es menor que la del cache. Un ancho de banda insuficiente en la RAM puede crear un cuello de botella cuando la CPU necesita grandes cantidades de datos rápidamente.

Cache: La cache tiene un ancho de Banda excepcionalmente alto, ya que a menudo está integrada directamente en el chip del CPU o muy cerca de él utilizando conexiones muy cortas o rápidas. Esto permite un flujo de datos casi instantáneo entre la cache y el núcleo del CPU, crucial para mantener el rendimiento del procesador al máximo.

En Resumen: Para la CPU, la **baja latencia** y el **alto ancho de banda** de la cache son vitales para reducir las demoras y evitar que el CPU tenga que esperar por los datos que vienen de la RAM más lenta.

c) ¿Qué es la localidad en el contexto de Memoria Cache?

La localidad (o principio de localidad de referencia) es un principio fundamental que justifica la existencia y la eficiencia de la memoria cache. Se basa en la observación de que durante la ejecución de un programa, el acceso a la memoria tiende a concentrarse en ciertas áreas.

Existen 2 tipos principales de localidad:

1.- Localidad Temporal

- Si se accede a un ítem de memoria en un momento dado, es muy probable que se vuelva a acceder a él en un futuro cercano.
- Ejemplos: Las variables de un bucle se usan repetidamente. La cache almacena estos datos la primera vez que se acceden, de modo que los accesos posteriores son ultra rápidos.

2. Localidad Espacial

- Si se accede a un ítem de Memoria en un momento dado, es muy probable que se acceda a ítems de Memoria cercanos a él en un futuro cercano.

Ejemplos: Cuando se acceda a un elemento de un array o una estructura de datos, el CPU probablemente necesite los elementos contiguos. Por ello, la cache no solo trae el dato solicitado, sino también un bloque de datos adyacentes para anticipar los próximos accesos.

Resumiendo

La cache funciona eficientemente porque explota este principio de localidad. Cuando el CPU solicita un dato, es probable que ese dato ya este en la cache (debido a la localidad temporal) o que el siguiente dato que necesitara este junto a él en la cache (debido a la localidad espacial).

6. Arquitectura y Procesadores Indicar si las siguientes afirmaciones son Verdaderas o Falsas, justificando cada respuesta:

Arquitectura:

- a. Las instrucciones en CISC no son uniformes en tamaño y no se ejecutan en ciclos fijos.
- b. Los procesadores RISC, al tener menos instrucciones y una menor complejidad en su decodificación y ejecución, generalmente consumen menos energía para ejecutar las mismas tareas en comparación con los procesadores CISC.

Pipeline:

- c. Pipeline es una técnica de diseño utilizada para mejorar la velocidad de procesamiento al permitir que varias instrucciones se ejecuten de manera secuencial (es decir, una después de la otra) en diferentes etapas del proceso de ejecución.

- d. Un riesgo en pipeline es cuando una instrucción depende del resultado de una instrucción anterior que aún no ha terminado de ejecutarse.

Performance:

- e. Un aumento en la frecuencia de reloj de un procesador siempre resulta en un aumento proporcional en el valor de MIPS, manteniendo constante el número de instrucciones por ciclo (IPC).
- f. Para los siguientes datos la computadora A tiene mejor performance que la computadora B porque ejecuta menos instrucciones en el mismo tiempo. Datos de la computadora A: Frecuencia de reloj: 3 GHz, CPI (Ciclos por Instrucción): 1,5. Datos de la computadora B: Frecuencia de reloj: 2,5 GHz, CPI (Ciclos por Instrucción): 1,0.

7. Componentes - Memoria Caché

- ¿Cuál es la diferencia principal entre la memoria RAM y la memoria caché en términos de velocidad y propósito dentro de un sistema computacional?
- ¿De qué manera la latencia y el ancho de banda afectan el rendimiento de la memoria RAM en comparación con la memoria caché?
- ¿Qué es la "localidad" en el contexto de la memoria caché?

a. ¿Cuál es la diferencia principal entre la memoria RAM y la memoria caché en términos de velocidad y propósito dentro de un sistema computacional?

Característica	Memoria RAM (Random Access Memory)	Memoria Caché (Cache Memory)
Velocidad	Relativamente lenta comparada con la caché.	Extremadamente rápida . Es la memoria más rápida del sistema, después de los registros del CPU.
Propósito	Almacenar temporalmente los datos y programas que el CPU está utilizando o necesitará pronto. Sirve como el área de trabajo principal del sistema.	Almacenar una pequeña porción de los datos y las instrucciones más frecuentemente usadas que residen en la RAM. Su objetivo es reducir el tiempo que el CPU debe esperar por los datos, actuando como un intermediario ultrarrápido.
Tamaño Típico	Grande (varios GB).	Pequeño (varios KB o MB).

 Exportar a Hojas de cálculo



b)

- **Latencia (Latency):**

- **RAM:** La latencia de la RAM es **significativamente mayor** que la de la caché. Una latencia alta significa que el CPU debe **esperar más ciclos de reloj** para que los datos sean recuperados de la RAM. Esto es un factor limitante en el rendimiento general del sistema.
- **Caché:** La caché está diseñada para tener una **latencia extremadamente baja**. Esto minimiza el tiempo de espera del CPU, permitiendo que las operaciones se realicen con mucha más rapidez, mejorando drásticamente el rendimiento cuando los datos requeridos están en la caché (un *cache hit*).

- **Ancho de Banda (Bandwidth):**

- **RAM:** El ancho de banda de la RAM es el volumen total de datos que puede transferirse en un periodo de tiempo. Si bien la RAM moderna tiene un ancho de banda alto, es **menor** que el de la caché. Un ancho de banda insuficiente en la RAM puede crear un **cuello de botella** cuando el CPU necesita grandes cantidades de datos rápidamente.
- **Caché:** La caché tiene un **ancho de banda excepcionalmente alto**, ya que a menudo está integrada directamente en el chip del CPU o muy cerca de él, utilizando conexiones muy cortas y rápidas. Esto permite un flujo de datos casi instantáneo entre la caché y el núcleo del CPU, crucial para mantener el rendimiento del procesador al máximo.

En resumen: Para el CPU, la **baja latencia** y el **alto ancho de banda** de la caché son vitales para reducir las demoras y evitar que el CPU tenga que esperar por los datos que vienen de la RAM más lenta.

c. ¿Qué es la "localidad" en el contexto de la memoria caché?

La **localidad (o principio de localidad de referencia)** es un principio fundamental que justifica la existencia y la eficacia de la memoria caché. Se basa en la observación de que, durante la ejecución de un programa, el acceso a la memoria tiende a concentrarse en ciertas áreas.

Existen dos tipos principales de localidad:

1. Localidad Temporal (Temporal Locality):

- Si se accede a un ítem de memoria en un momento dado, es **muy probable que se vuelva a acceder a él en un futuro cercano**.
- *Ejemplo:* Las variables de un bucle se usan repetidamente. La caché almacena estos datos la primera vez que se acceden, de modo que los accesos posteriores son ultrarrápidos.

2. Localidad Espacial (Spatial Locality):

- Si se accede a un ítem de memoria en un momento dado, es **muy probable que se acceda a ítems de memoria cercanos a él en un futuro cercano**.
- *Ejemplo:* Cuando se accede a un elemento de un *array* o una estructura de datos, el CPU probablemente necesitará los elementos contiguos. Por ello, la caché no solo trae el dato solicitado, sino también un bloque de datos adyacentes (*cache line*) para anticipar los próximos accesos.

La caché funciona eficientemente porque **explota** este principio de localidad. Cuando el CPU solicita un dato, es probable que ese dato ya esté en la caché (debido a la localidad temporal) o que el siguiente dato que necesitará esté junto a él en la caché (debido a la localidad espacial).

6. Arquitectura y Procesadores Indicar si las siguientes afirmaciones son Verdaderas o Falsas, justificando cada respuesta:

Arquitectura:

a. Las instrucciones en CISC no son uniformes en tamaño y no se ejecutan en ciclos fijos.

b. Los procesadores RISC, al tener menos instrucciones y una menor complejidad en su decodificación y ejecución, generalmente consumen menos energía para ejecutar las mismas tareas en comparación con los procesadores CISC.

Pipeline:

c. Pipeline es una técnica de diseño utilizada para mejorar la velocidad de procesamiento al permitir que varias instrucciones se ejecuten de manera secuencial (es decir, una después de la otra) en diferentes etapas del proceso de ejecución.

a) Las instrucciones en CISC no son uniformes en tamaño y no se ejecutan en ciclos fijos

Verdadera

- CISC (Complex Instruction Set Computing) Se caracteriza por tener un conjunto de instrucciones muy amplio y complejo.
- No uniformes en tamaño: Las instrucciones CISC pueden variar mucho en longitud (Por ejemplo desde 1 byte hasta 15 bytes o más), ya que una sola instrucción puede realizar múltiples operaciones (como cargar, operar y almacenar).
- No se ejecutan en ciclos fijos: Debido a su complejidad variable, diferentes instrucciones CISC requieren un número diferente de ciclos de reloj para completarse. Las instrucciones más complejas pueden necesitar muchos ciclos, mientras que las simples necesitan menos. Esto contrasta con RISC donde la mayoría de las instrucciones se diseñan para ejecutarse en un solo ciclo de reloj.

Respuesta: Verdadera (V)

Justificación:

- **CISC (Complex Instruction Set Computing)** se caracteriza por tener un conjunto de instrucciones muy amplio y complejo.
- **No Uniformes en Tamaño:** Las instrucciones CISC pueden variar mucho en longitud (por ejemplo, desde 1 byte hasta 15 bytes o más), ya que una sola instrucción puede realizar múltiples operaciones (como cargar, operar y almacenar).
- **No se Ejecutan en Ciclos Fijos:** Debido a su complejidad variable, diferentes instrucciones CISC requieren un número diferente de ciclos de reloj para completarse. Las instrucciones más complejas pueden necesitar muchos ciclos, mientras que las simples necesitan menos. Esto contrasta con RISC, donde la mayoría de las instrucciones se diseñan para ejecutarse en un solo ciclo de reloj.

b) Los procesadores RISC, al tener menos instrucciones y una menor complejidad en su decodificación y ejecución, generalmente consumen menos energía para ejecutarse los mismos tareas en comparación con los procesadores CISC

Verdadero

Justificación:

Respuesta: Verdadera (V)

Justificación:

- **RISC (Reduced Instruction Set Computing)** utiliza un conjunto de instrucciones más simple y pequeño.
- **Menor Complejidad Hardware:** Las instrucciones RISC son uniformes y se ejecutan rápidamente. Esto permite que la unidad de control y los circuitos de decodificación dentro del CPU sean mucho más simples.
- **Menor Consumo de Energía:** La simplicidad del *hardware* (menos transistores para la lógica de control compleja) y la ejecución más rápida y predecible de las instrucciones se traducen en un **menor calor generado y un menor consumo de energía** en comparación con la lógica más densa y compleja de los procesadores CISC. Esto hace que RISC sea dominante en dispositivos móviles y sistemas embebidos (como *smartphones* y tabletas), donde la eficiencia energética es crucial.

Pipeline :

c) Pipeline es una técnica de diseño utilizada para mejorar la velocidad de procesamiento al permitir que varias instrucciones se ejecuten de manera secuencial (es decir, una después de la otra) en diferentes etapas del proceso de ejecución. Respuesta Falsa

Justificación:

- La afirmación describe correctamente el concepto de **pipeline (segmentación)** como una técnica para mejorar la velocidad al dividir la ejecución de una instrucción en etapas (*fetch, decode, execute, writeback*).
- Sin embargo, el error está en el término "**de manera secuencial (es decir, una después de la otra)**". El **pipeline no hace que las instrucciones se ejecuten una después de la otra** (lo que sería la ejecución no segmentada o serial).
- **Lo que hace el **pipeline** es permitir que la ejecución de **varias instrucciones se superpongan o ejecuten concurrentemente** en el tiempo, a través de las diferentes etapas, como en una línea de ensamblaje (o tubería).
 - Mientras la Instrucción 1 está en la etapa de Ejecución, la Instrucción 2 puede estar en la etapa de Decodificación y la Instrucción 3 en la etapa de Búsqueda.
- La ejecución "**secuencial**" se refiere a la **finalización en orden** pero no a la ejecución de sus etapas. Lo esencial del **pipeline** es la **conurrencia o paralelismo a nivel de instrucción para lograr un mayor rendimiento** (tasa de *throughput*) global.

d. Un riesgo en pipeline es cuando una instrucción depende del resultado de una instrucción anterior que aún no ha terminado de ejecutarse.

Performance:

e. Un aumento en la frecuencia de reloj de un procesador siempre resulta en un aumento proporcional en el valor de MIPS, manteniendo constante el número de instrucciones por ciclo (IPC).

f. Para los siguientes datos la computadora A tiene mejor performance que la computadora B porque ejecuta menos instrucciones en el mismo tiempo. Datos de la computadora A: Frecuencia de reloj: 3 GHz, CPI (Ciclos por Instrucción): 1,5. Datos de la computadora B: Frecuencia de reloj: 2,5 GHz, CPI (Ciclos por Instrucción): 1,0.

d) Un riesgo en pipeline es cuando una instrucción depende del resultado de una instrucción anterior que aún no ha terminado de ejecutarse.

Verdadero

Justificación:

- Esta situación se conoce específicamente como un **Riesgo de Datos (Data Hazard)**.
- **Dependencia:** Ocurre cuando la instrucción I_2 necesita el resultado de la instrucción I_1 , pero I_1 todavía no ha escrito ese resultado de vuelta al registro (*writeback*).
- **Problema en Pipeline:** Dado que el pipeline superpone las etapas, I_2 intentará leer el resultado de I_1 antes de que I_1 lo haya producido. Esto puede llevar a un resultado incorrecto.
- **Solución:** Se utilizan técnicas como el **reenvío (forwarding)** o la **inserción de burbujas (stalls/bubbles)** para resolver este riesgo y asegurar que la instrucción posterior obtenga el valor correcto.

Performance:

e) Un aumento en la frecuencia de reloj de un procesador siempre resulta en un aumento proporcional en el valor de MIPS, manteniendo constante el número de instrucciones por ciclo (IPC)

Falso

$$MIPS = \frac{FR}{(CPI \times 10^6)}$$

$$IPC = \frac{1}{CPI}$$

$$MIPS = \frac{FR \times IPC}{10^6}$$

Justificación:

- **Fórmula de MIPS:** $MIPS = \text{Frecuencia de Reloj} / (CPI \times 10^6)$, donde $IPC = 1/CPI$. Por lo tanto, $MIPS = \text{Frecuencia de Reloj} \times IPC / 10^6$.
- **Aumento Proporcional:** Matemáticamente, si el IPC se mantiene constante, un aumento en la frecuencia de reloj sí resulta en un aumento proporcional en MIPS.
- **¿Por qué es Falsa?:** En la práctica y el diseño de procesadores, augmentar la frecuencia de reloj a menudo obliga al diseñador a aumentar el CPI (es decir, reducir el IPC). → CPI
 - Las frecuencias más altas requieren que las señales eléctricas viajen más rápido. Esto obliga a dividir las etapas del pipeline en partes más pequeñas (añadir más etapas), lo que augmenta la latencia y el CPI efectivo de las instrucciones.
 - Además, a frecuencias muy altas, los riesgos de pipeline y los fallos de caché se hacen más costosos en términos de ciclos perdidos, lo que reduce el IPC real.
 - Por lo tanto, un aumento de frecuencia **no siempre** garantiza un aumento proporcional en el rendimiento real (MIPS/IPC), ya que puede introducir penalizaciones que afectan negativamente al IPC.

f) Para las siguientes datos de la computadora A tiene mejor performance que la computadora B porque ejecuta menos instrucciones en el mismo tiempo. Datos de la computadora A: Frecuencia de Reloj: 3GHz, CPI (Ciclo por Instrucción): 1.5

Datos de la computadora B: Frecuencia de Reloj: 2.5GHz, CPI (Ciclo por Instrucción): 1.0

Falsa

Justificación:

- **Error en la Razón:** La razón dada ("ejecuta menos instrucciones en el mismo tiempo") es una conclusión sin base en los datos y **no es la métrica de rendimiento correcta**. Para comparar el rendimiento, debemos usar la métrica **MIPS (Millones de Instrucciones por Segundo)**.
- **Cálculo de MIPS (Instrucciones por Segundo):**

El rendimiento se calcula como:

$$MIPS = \frac{\text{Frecuencia de Reloj (MHz)}}{CPI}$$

Computadora A:

- Frecuencia: 3 GHz = 3000 MHz
- CPI: 1.5

$$MIPS_A = \frac{3000 \text{ MHz}}{1.5} = 2000 \text{ MIPS}$$

$$MIPS = \frac{FR}{CPI} = \frac{3000}{1.5}$$

2000 MIPS

Computadora B:

- Frecuencia: 2.5 GHz = 2500 MHz
- CPI: 1.0

$$MIPS_B = \frac{2500 \text{ MHz}}{1.0} = 2500 \text{ MIPS}$$

$$MIPS = \frac{FR}{CPI} = \frac{2500}{1.0} = 2500 \text{ MIPS}$$

Conclusión: La Computadora B tiene un valor de **2500 MIPS**, mientras que la Computadora A tiene **2000 MIPS**.

Dado que B puede ejecutar un mayor número de instrucciones por segundo, **la Computadora B tiene mejor rendimiento** que la Computadora A. La afirmación original es, por lo tanto, **Falsa**.

La computadora B puede ejecutar mayor número de instrucciones que la computadora A.

- 2) a. Representar según la norma IEEE754 el número -45,31 (base 10).
 b. Convertir a decimal el punto flotante 0100001101010110011000000000

a) I) Analizar el signo : Negativo $\Rightarrow$ (1)

II) Pasar a Binario 45,31

$$\begin{array}{r} 45 \overline{) 16} \\ 32 \quad 2 \\ \hline 13 \quad 2 \\ \hline 8 \end{array}$$

$$45_{[10]} = 2D_{[16]} = 00101101_{[2]}$$

Ahora los decimales
0,31

$$\begin{aligned} 0,31 \times 16 &= 4,96 \\ 0,96 \times 16 &= F,36 \\ 0,36 \times 16 &= 5,76 \\ 0,76 \times 16 &= C,16 \\ 0,16 \times 16 &= 2,56 \\ 0,56 \times 16 &= 8,96 \\ 0,96 \times 16 &= F,36 \end{aligned}$$

$$0,31_{[10]} = 0,4F5C28F..._{[16]}$$

$$0,010011110101110000121000$$

$$00101101010011110101110000121000_{[2]}$$

III) Normalizar

$$1,01101010011110101110000121000 \times 2^5$$

IV) Exponente con Exceso 127

$$127 + 5 = 128 + 4$$

$$10000000 + 100$$

$$\boxed{10000100}$$

Representación IEEE754

$$1 \mid 10000100 \mid 0110101001111010111000$$

- 2) a. Representar según la norma IEEE754 el número -45,31 (base 10).
 b. Convertir a decimal el punto flotante 01000011010101100110000000000

b) I) Pasar a Binario

01000011010101100110000000000

II) Separar en Grillo

Signo: 0 (Positive)

Exponente: 10000110

Mantisa:

1.101011011001100000000000

III) Exponente con Exceso 127

10000110

100000000 + 110

128 + 6

127 + 7 $\Rightarrow$ Exponente: 7

IV) Convertir a Decimal la Mantisa

0 -1 -3 -5 -6 -8 -9 -12 -13
 1.1010110110011

$2^0 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-6} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13}$

1.678100586

V) Juntar Todo

1.678100586×2^7

b)

214.796875 [10]